Stock Portfolio Risk & Forecasting Platform

Business Requirements Document

A Django REST Framework backend with JWT-secured, role-based portfolio management, a machine-learning risk and forecast engine, admin web pages for data and model training, and a Flutter client used to test the platform on phones, emulators and the browser.

Prepared by: Dion James Smith
Version 2.0 — November 2026
Supersedes Version 1.0 (September 2026)

Table of Contents

Document Changes from Version 1.0 4

1. Executive Summary 4

2. Project Objectives 4

3. Technology Stack 5

4. System Architecture Overview 5

4.1 Layers . . . 5

4.2 Clients . . . 6

4.3 Request flow . . . 6

4.4 Background flow . . . 6

5. Application (Django App) Structure 6

6. Database Design 7

6.1 Entity overview . . . 7

6.2 users . . . 7

6.3 portfolios . . . 7

6.4 holdings . . . 8

6.5 price_history . . . 8

6.6 dataset_uploads . . . 8

6.7 training_runs . . . 9

6.8 Relationships . . . 9

7. Authentication and Account Creation 9

7.1 Custom user model . . . 9

7.2 How each role is created . . . 9

7.3 Token lifecycle . . . 10

7.4 Token lifetimes and limits . . . 10

7.5 Authorization model . . . 10

8. Roles and Permission Matrix 10

9. API Endpoint Reference 11

9.1 Accounts — /api/accounts/ . . . 11

9.2 Portfolios and holdings — /api/ . . . 11

9.3 Market data — /api/market/ . . . 11

9.4 Analytics — /api/analytics/ . . . 12

9.5 Dashboards — /api/dashboard/ . . . 12

10. API Response Contract 12

11. Django Web Interface 13

11.1 Dataset upload requirements . . . 13

11.2 Model training requirements . . . 13
12. Flutter Test Client Requirements 14
12.1 Purpose and scope . . . 14
12.2 Test targets and base URLs . . . 14
12.3 Backend requirements for Flutter . . . 14
12.4 App architecture . . . 15
12.5 Authentication and session requirements . . . 15
12.6 Screen inventory — common . . . 15
12.7 Screen inventory — investor . . . 16
12.8 Screen inventory — analyst . . . 16
12.9 Screen inventory — admin . . . 16
12.10 Page design requirements . . . 17
12.11 Acceptance criteria for the Flutter test . . . 17
13. Celery and Redis Task Design 17
13.1 Redis databases . . . 17
13.2 Scheduled and on-demand tasks . . . 18
13.3 Caching . . . 18
14. Machine Learning Pipeline 18
14.1 Code location . . . 18
14.2 Features . . . 18
14.3 Model 1 — Risk classification . . . 19
14.4 Model 2 — Direction forecast . . . 19
14.5 Training method and evaluation . . . 19
14.6 Versioning . . . 19
15. Dataset Source and Ingestion 19
16. Environment Configuration 20
17. Project Directory Structure 20
18. Security and Non-Functional Requirements 21
19. Delivery Roadmap and Future Scope 21
Appendix A — Sample Flutter Test Flow 22
Appendix B — Machine Learning Disclaimer 22

Document Changes from Version 1.0

Area	Version 1.0	Version 2.0
Apps	accounts, core, market_data, portfolios, analytics (+ config folder)	accounts, core (config merged in), market_data, portfolios, ml_training (new), analytics, dashboards
Machine-learning code	Inside analytics	All training code in ml_training; analytics only serves predictions
Dataset loading	Command line (load_price_history)	Datasets web page: CSV files, folder or Kaggle zip, processed by Celery with progress
Model training	Command line (train_models)	ML Training web page for admins, with run history and model version pages
Database	4 tables	6 tables: adds dataset_uploads and training_runs
Clients	Flutter planned for a later phase	Flutter app is the primary test client; screens, design rules and network setup are specified (Section 12)
Views	ViewSets and routers	Function-based views; forms validate input, serializers shape output

1. Executive Summary

This platform lets investors track stock portfolios, view stored market data, and receive machine-learning risk classifications and next-day price direction forecasts for each holding. It is built with Django and Django REST Framework, secured with a custom JWT scheme on a purpose-built User model, and organised around three roles — investor, analyst and admin — each with distinct data access rules.

Two supervised models sit at the core of the analytics layer: a Random Forest classifier that labels each holding's risk as low, medium or high, and a Gradient Boosting classifier that predicts whether the next trading day's close will be up or down. Each model is compared against simple baselines so its value can be judged honestly.

Admins control the data and the models from Django web pages: they upload the historical price dataset, start training runs, follow progress, and review every saved model version. Celery and Redis handle all slow work in the background — dataset imports, daily price updates, nightly predictions, weekly retraining — and Redis also caches expensive summaries and enforces rate limits. PostgreSQL is the system of record, accessed only through the Django ORM.

The project will be tested through a Flutter app. Every investor, analyst and admin feature that Flutter needs is available through the JSON API, and Section 12 specifies the Flutter screens, how they map to API endpoints, how they must be designed for phones first, and the backend settings needed for an emulator, a physical phone and Chrome to reach the API.

2. Project Objectives

- **Custom authentication** — a dedicated accounts app with its own User model, JWT access and refresh tokens built with PyJWT, logout blacklisting and role-based, object-level authorization.
- **Relational database design** — normalised tables with clear foreign keys, including a self-referencing analyst assignment, and ORM joins and aggregations instead of duplicated fields.
- **Explainable machine learning** — two well-understood algorithms with leak-free features, time-based validation and baseline comparison, integrated into live APIs and background tasks.
- **Admin control through the GUI** — dataset upload and model training are done from Django templates, not the command line.

- **Asynchronous design** — Celery and Redis for scheduled ingestion, predictions, retraining, uploads, caching and throttling.
- **API-first design for Flutter** — consistent JSON responses so a Flutter app can be built and tested against the backend without backend changes.

3. Technology Stack

Layer	Technology	Purpose
Web / API framework	Django 6.1 + Django REST Framework 3.18	Function-based API views, routing, serializers for output
Web interface	Django templates, Bootstrap 5, Chart.js	Session-based pages, admin dataset and training pages
Database	PostgreSQL (psycopg 3)	Users, portfolios, holdings, price history, uploads, training runs
Data access	Django ORM	All reads and writes; no raw SQL
Authentication	Custom JWT with PyJWT	Access and refresh tokens, each signed with its own secret
Task queue	Celery 5.6 + Celery Beat	Scheduled and on-demand background work
Broker / cache	Redis	Broker, results, cache, throttles and token blacklist (separate databases)
Machine learning	scikit-learn, pandas, NumPy, joblib	Random Forest, Logistic Regression baseline, Gradient Boosting
Market data	Kaggle CSV dataset, yfinance	Ten-year history and daily updates
Test client	Flutter (Android, iOS, web)	Primary client used to test the platform (Section 12)
Web server (deployment)	Gunicorn	Serving the Django app in containers (Phase 12)

4. System Architecture Overview

4.1 Layers

- **Client layer** — the Flutter app (JWT), the Django web pages (session login) and the Postman collection.
- **API and web layer** — function-based DRF API views and Django template views. JWT authentication and permission checks run before any business logic.
- **Business logic layer** — forms and ModelForms validate input; services and selectors hold the logic; serializers shape output.
- **Data layer** — PostgreSQL through the ORM, with select_related, prefetch_related and annotate to keep queries efficient.
- **Asynchronous layer** — Celery workers and Celery Beat with Redis, for imports, price updates, predictions, training and cache invalidation.

4.2 Clients

Client	Who uses it	How it connects	What it covers
Flutter app	Investors, analysts, admins	JSON API with Bearer JWT	Everything except file upload and starting training
Django web pages	All roles; admin pages for admins	Browser session login	Dashboards, portfolios, users, Datasets, ML Training, Model versions
Postman collection	Developer	JSON API with Bearer JWT	Automated end-to-end API check

4.3 Request flow

Client → JWT authentication (or session) → role and object-level checks → view → form validation → service / ORM → PostgreSQL → serializer → standard JSON response. Summary and dashboard endpoints read Redis first and fall back to the database.

4.4 Background flow

Celery Beat and user actions put jobs on the Redis queue. Workers import uploaded files, fetch prices, compute predictions and train models, write results through the ORM, save models to the registry folder, and clear the related cache entries so the next request shows fresh data. Pages that start a job show its progress and refresh themselves.

5. Application (Django App) Structure

App	Owns	Responsibility
accounts	User	Registration, login, JWT tokens, logout, profile, password change, role and analyst assignment, account web pages
core	—	Shared code: logger, decorators (@log_execution_time, @admin_required), api_response, permissions, pagination, exception handler, throttles, cache keys, multiple-file field, ping task
market_data	PriceHistory, DatasetUpload	Dataset upload pages and import task, Yahoo Finance updates, latest price, history and refresh APIs
portfolios	Portfolio, Holding	Role-scoped CRUD APIs, portfolio summary
ml_training	TrainingRun	Algorithms, features, labels, data split, training, evaluation, model registry, training and model version pages
analytics	—	Loads the newest models, writes predictions to holdings, risk and forecast APIs, portfolio risk summary, task status
dashboards	—	Investor, analyst and admin dashboards (API and web), holding trend, portfolio and holding pages

The config folder of version 1.0 has been merged into core so shared code lives in one place. The ml_training app keeps every part of model building together; analytics only uses the saved models.

6. Database Design

6.1 Entity overview

Table	App	Represents
users	accounts	Every person: investor, analyst or admin
portfolios	portfolios	A named collection of holdings owned by one user
holdings	portfolios	One stock position plus its latest ML outputs
price_history	market_data	Daily OHLCV prices per ticker, shared by every holding of that ticker
dataset_uploads	market_data	One dataset upload: status, progress and per-file results
training_runs	ml_training	One training run: options, status, current step, new model versions and results

6.2 users

Field	Type	Notes
id	Integer	Primary key
username, email	String	Unique (case-insensitive checks)
password	Hashed string	Never stored or returned in plain text
role	Choice	investor / analyst / admin; admin also sets is_staff and is_superuser
assigned_analyst_id	FK to users.id	Nullable; only investors keep one; must be an active analyst
is_active, date_joined, updated_at	Boolean / DateTime	Standard account fields

6.3 portfolios

Field	Type	Notes
id	Integer	Primary key
user_id	FK to users.id	Owner
name	String	Required; unique per owner
description	Text	Optional
created_at, updated_at	DateTime	

6.4 holdings

Field	Type	Notes
id, portfolio_id	Integer / FK	Belongs to one portfolio; deleted with it
ticker	String	Stored uppercase; cannot be changed after creation; joined to price_history
quantity, buy_price	Decimal	Must be greater than zero
buy_date	Date	Cannot be in the future
risk_label, risk_score	Choice / Float	low / medium / high and its probability; written only by tasks
predicted_direction, prediction_confidence	Choice / Float	up / down and its probability; written only by tasks
last_predicted_at	DateTime	Time of the latest prediction
risk_model_version, forecast_model_version	String	Model versions that produced the outputs
created_at, updated_at	DateTime	

6.5 price_history

Field	Type	Notes
ticker, date	String / Date	Unique together
open_price, high_price, low_price, close_price	Decimal (12,4)	Validated: positive, high ≥ low
volume	BigInteger	Nullable when the source has no volume
created_at	DateTime	

price_history has no foreign key to holdings: prices belong to a ticker and are shared by every investor holding it. They are joined at query time on the ticker.

6.6 dataset_uploads

Field	Type	Notes
uploaded_by_id	FK to users.id	Admin who uploaded; kept as null if the user is deleted
name, tickers	String	Summary of the files; optional ticker filter
status	Choice	pending / processing / completed / failed
total_files, files_processed	Integer	Drive the progress bar
tickers_loaded, rows_inserted, rows_skipped	Integer	Result counts
results	JSON	One entry per file: ticker, status, rows added and skipped, message
error, task_id, created_at, started_at, finished_at	Text / String / DateTime	

6.7 training_runs

Field	Type	Notes
model_choice	Choice	all / risk / forecast
tickers, update_predictions	String / Boolean	Optional ticker filter; whether to recompute holdings afterwards
triggered_by_id	FK to users.id	Null for the weekly scheduled run
status, current_step	Choice / String	pending / running / completed / failed and the step in progress
risk_version, forecast_version	String	Versions saved by this run
result, error	JSON / Text	Metric summaries, prediction counts, error message
task_id, created_at, started_at, finished_at	String / DateTime	

6.8 Relationships

- users (1) — (many) portfolios (1) — (many) holdings — matched by ticker — price_history.
- users (1) — (many) users through assigned_analyst_id (analyst to investors).
- users (1) — (many) dataset_uploads and training_runs.

7. Authentication and Account Creation

7.1 Custom user model

The project defines its own User model from the first migration, with role and assigned_analyst as real fields, and sets it as AUTH_USER_MODEL.

7.2 How each role is created

Public registration always creates an investor. Nobody chooses their own role; a role field in the request is ignored. This prevents anyone from signing up as an analyst or admin.

Role	How it is created
Investor	POST /api/accounts/register/ or the Register page
Analyst	Register normally, then an admin sets role to analyst (PATCH /api/accounts/users/{id}/ or the Users page)
Admin	python manage.py createsuperuser, or an admin sets role to admin

Registration request body (same for every new user):

```
POST /api/accounts/register/
{
    "username": "dion_investor",
    "email": "dion.investor@example.com",
    "password": "Investor-Pass-2026!",
    "password_confirmation": "Investor-Pass-2026!"
}
```

Promotion and assignment by an admin (Authorization: Bearer <admin access token>):

```
PATCH /api/accounts/users/{analyst_id}/      {"role": "analyst"}
PATCH /api/accounts/users/{investor_id}/     {"assigned_analyst": <analyst_id>}
```

- An admin cannot change their own role.
- Changing an analyst to another role unassigns all of their investors.

- A user who stops being an investor loses their assigned analyst.

7.3 Token lifecycle

- **Login** with username or email returns the user (including role) and an access/refresh token pair.
- **Requests** send Authorization: Bearer <access token>.
- **Refresh** exchanges a valid refresh token for a new access token.
- **Logout** blacklists the refresh token in a separate Redis database until it would have expired.
- **Password change** invalidates every older token and returns a new pair.
- Access and refresh tokens use different secrets and carry a token_type claim, so one cannot be used as the other.

7.4 Token lifetimes and limits

Item	Value
Access token	15 minutes
Refresh token	7 days
Login rate limit	5 per minute per IP
Registration rate limit	10 per hour per IP
Price refresh rate limit	10 per hour per user

7.5 Authorization model

- **Role checks** — some endpoints are limited to a role (for example only admins manage users).
- **Object-level checks** — querysets are filtered so investors see only their own records and analysts only their assigned investors' records. A record outside the user's scope returns 404, which prevents guessing other users' IDs. A visible record the user may not change returns 403.
- **Admin web pages** — the @admin_required decorator protects the Datasets, ML Training and Users pages and sends other roles back to their dashboard.

8. Roles and Permission Matrix

Action	Investor	Analyst	Admin
Register, log in, edit own profile	Yes	Yes	Yes
View own portfolios and holdings	Yes	—	Yes
View assigned investors' portfolios	No	Yes (read-only)	Yes
View all portfolios	No	No	Yes
Create / edit / delete portfolios and holdings	Own only	No	Yes
View prices and price history	Yes	Yes	Yes
Queue a price refresh	No	Yes	Yes
View risk, forecast, risk summary, trend	Own	Assigned	All
Queue a prediction recompute, check task status	No	Yes	Yes
Investor / analyst / admin dashboard	Investor	Analyst	Admin
List users, change roles, assign analysts	No	No	Yes
Upload datasets, train models, view model versions (web)	No	No	Yes

9. API Endpoint Reference

All endpoints are under /api/. Every response uses the format in Section 10.

9.1 Accounts — /api/accounts/

Method	Endpoint	Access	Purpose
POST	register/	Public	Create an investor
POST	login/	Public	username_or_email + password → user + tokens
POST	token/refresh/	Refresh token	New access token
POST	logout/	Authenticated	Blacklist the refresh token
GET / PATCH	me/	Authenticated	Own profile; PATCH username / email
POST	change-password/	Authenticated	New password and new token pair
GET	users/	Admin	Paginated user list
PATCH	users/{id}/	Admin	Change role or assigned analyst

9.2 Portfolios and holdings — /api/

Method	Endpoint	Access	Purpose
GET / POST	portfolios/	All (scoped) / investor, admin	List visible portfolios; create (admin may pass user_id)
GET / PATCH / DELETE	portfolios/{id}/	Visible / owner, admin	Detail, rename, delete
GET	portfolios/{id}/summary/	Visible	Cost, value, gain/loss, risk distribution, per-holding rows
GET / POST	portfolios/{id}/holdings/	Visible / owner, admin	List or add holdings
GET / PATCH / DELETE	holdings/{id}/	Visible / owner, admin	Holding detail, edit, delete

9.3 Market data — /api/market/

Method	Endpoint	Access	Purpose
GET	prices/{ticker}/	Authenticated	Latest stored price
GET	prices/{ticker}/history/?start=&end;=	Authenticated	Paginated history, newest first
POST	refresh/{ticker}/	Analyst, admin	Queue a price download (202 + task_id)

9.4 Analytics — /api/analytics/

Method	Endpoint	Access	Purpose
GET	holdings/{id}/risk/	Visible	Risk label, score, model version, holding age, gain/loss %
GET	holdings/{id}/forecast/	Visible	Direction, confidence, model version
POST	holdings/{id}/recompute/	Analyst, admin	Queue a prediction (202 + task_id)
GET	portfolio/{id}/risk-summary/	Visible	Risk and direction distribution, average confidence, total value
GET	tasks/{task_id}/	Analyst, admin	Celery state and result

9.5 Dashboards — /api/dashboard/

Method	Endpoint	Access	Purpose
GET	investor/	Investor	Portfolios, totals, risk distribution
GET	analyst/	Analyst	Clients, totals, high-risk holdings
GET	admin/	Admin	User counts, totals, latest price date, models in use
GET	holdings/{id}/trend/?start=&end;=	Visible	Price points for a chart (default last 180 days)

10. API Response Contract

Every API response — success or error — has the same four keys. Flutter reads the same shape everywhere.

```
{
    "status": "success" | "error",
    "message": "Human-readable message",
    "data": { ... } | [ ... ] | null,
    "errors": null | { "field_name": ["Error message"] }
}
```

Topic	Rule
Pagination	data contains count, next, previous and results; 20 per page, ?page_size= up to 100
Decimals	Prices, quantities and money values are strings (for example "150.0000") to avoid rounding errors
Dates and times	Dates are YYYY-MM-DD; date-times are ISO 8601 in UTC
Cache flag	Summary, risk-summary and dashboard responses include from_cache
Background jobs	202 Accepted with data.task_id

Status code	Meaning	What the client should do
200 / 201	Success / created	Show the data
202	Job queued	Show 'in progress' and poll the task status
400	Validation failed	Show errors under each form field
401	Token missing, invalid or expired	Refresh once and retry; if that fails, go to login
403	Role not allowed	Hide the action or show a 'not permitted' message
404	Not found or not visible to this user	Show a not-found message

Status code	Meaning	What the client should do
429	Rate limit	Wait errors.retry_after_seconds before retrying
500 / 503	Server error / task queue unavailable	Show a retry option

11. Django Web Interface

The web interface uses session login and the same role rules as the API. The admin pages below replace the command-line tools of version 1.0: an admin controls data and training entirely from the browser.

URL	Who	Page
/, /register/, /login/	Everyone	Home and account pages
/dashboard/	All roles	Role-specific dashboard
/profile/, /profile/password/	All roles	Profile and password change
/portfolios/, /portfolios/{id}/, /holdings/{id}/	All roles (scoped)	Portfolios, holdings, price chart
/users/	Admin	Role changes and analyst assignment
/datasets/, /datasets/{id}/	Admin	Upload CSV files, a folder or a zip; progress; per-file results; fetch latest prices
/ml/, /ml/runs/{id}/	Admin	Data readiness, start training, live run progress, cancel, history
/ml/models/, /ml/models/{type}/{version}/	Admin	Every model version with metrics, baselines, confusion matrix, feature importance
/admin/	Admin	Django admin

11.1 Dataset upload requirements

- Accept several CSV files, a whole folder, or the Kaggle zip as downloaded; ignore files that are not .csv or .zip.
- Each file is named after its ticker and has Date, Open, High, Low, Close and optionally Volume columns.
- Optional ticker filter.
- Import runs in Celery; the page shows files processed / total and refreshes itself.
- Already-stored rows are skipped, so repeat uploads are safe; uploaded files are deleted after import.
- Each file's result is shown: rows added, rows skipped, or the reason it failed.

11.2 Model training requirements

- Show whether stored prices cover the training, validation and test periods; training is disabled until they do.
- Train both models, the risk model only, or the forecast model only, optionally for selected tickers.
- Optionally update every holding's prediction when training finishes.
- Only one active run at a time; a run waiting for a worker can be cancelled.
- Show the current step while running and the test metrics against baselines when finished.
- Keep every model version; the newest of each type is used for predictions.

12. Flutter Test Client Requirements

The platform will be tested by running a Flutter app against the API. The Flutter pages must therefore be designed for the API exactly as it is, for phone screens first, and for each role separately. This section is the specification for that app and for the backend settings it needs.

12.1 Purpose and scope

- Exercise every investor and analyst feature, and the admin dashboard and user management, through the JSON API.
- Run on the Android emulator, a physical phone on the same Wi-Fi, the iOS simulator (on macOS) and Chrome.
- Built as a separate Flutter project; it never connects to the database directly.
- Dataset upload and starting training are done in the Django web pages; the Flutter admin links to them.

12.2 Test targets and base URLs

Target	Base URL used by Flutter	Backend requirement
Android emulator	http://10.0.2.2:8000	10.0.2.2 in ALLOWED_HOSTS
iOS simulator	http://127.0.0.1:8000	Already allowed
Physical phone (same Wi-Fi)	http://<computer IP>:8000	Computer IP in ALLOWED_HOSTS; runserver 0.0.0.0:8000
Flutter web (Chrome)	http://127.0.0.1:8000	Run Flutter web on port 3000 (already in CORS_ALLOWED_ORIGINS)

12.3 Backend requirements for Flutter

ID	Requirement
BE-01	ALLOWED_HOSTS includes localhost, 127.0.0.1, 10.0.2.2 and the computer's Wi-Fi IP during testing. With ALLOWED_HOSTS = [] Django rejects the emulator and phones with 400 Bad Request.
BE-02	CORS allows the Flutter web origin (http://localhost:3000 and http://127.0.0.1:3000). Native apps are not affected by CORS.
BE-03	Login and /api/accounts/me/ return role so the app can open the right screens.
BE-04	Every response follows Section 10, including validation errors per field.
BE-05	Long operations return 202 with a task_id that can be polled.
BE-06	The Celery app is imported in stock_risk_engine/__init__.py so queued jobs use the project's Celery settings.
BE-07 (optional)	Read-only admin APIs for dataset uploads, training runs and model versions, if the Flutter admin should show their status.

12.4 App architecture

Concern	Choice
HTTP	dio with an interceptor that adds the Bearer token and handles 401 by refreshing once
Token storage	flutter_secure_storage (never plain shared preferences)
State	provider for the logged-in user and role (simple and beginner-friendly)
Navigation	go_router with redirects by login state and role
Charts	fl_chart for price trends
Formatting	intl for money, percentages and dates
Configuration	--dart-define=API_BASE_URL=... per target
Android testing	android:usesCleartextTraffic="true" in debug builds so plain HTTP works locally

12.5 Authentication and session requirements

ID	Requirement
FL-01	Splash screen reads stored tokens, calls /api/accounts/me/ and opens the home screen for the user's role, or Login.
FL-02	Login accepts username or email; on success both tokens are stored securely.
FL-03	Register creates an investor and then shows Login (or logs in automatically).
FL-04	On 401 the app refreshes the access token once and retries; if refresh fails it clears tokens and opens Login.
FL-05	Logout calls /api/accounts/logout/ with the refresh token and clears storage.
FL-06	After a password change the new token pair from the response replaces the old one.
FL-07	Screens and buttons a role cannot use are not shown.

12.6 Screen inventory — common

Screen	API	Content
Splash	GET accounts/me/	Route by role
Login	POST accounts/login/	Username or email, password, link to Register
Register	POST accounts/register/	Username, email, password, confirm password
Profile	GET / PATCH accounts/me/	Username, email, role badge, assigned analyst, logout
Change password	POST accounts/change-password/	Old, new and confirm password

12.7 Screen inventory — investor

Screen	API	Content
Home	GET dashboard/investor/	Total value, gain/loss, portfolio count, risk distribution, portfolio cards
Portfolios	GET portfolios/	List with holdings count and value; pull-to-refresh; add button
Portfolio form	POST portfolios/, PATCH portfolios/{id}/	Name, description
Portfolio detail	GET portfolios/{id}/summary/, GET analytics/portfolio/{id}/risk-summary/	Value, cost, gain/loss, risk and direction distribution, holdings list
Holding form	POST portfolios/{id}/holdings/, PATCH holdings/{id}/	Ticker (create only), quantity, buy price, buy date
Holding detail	GET holdings/{id}/, analytics risk/ and forecast/, dashboard/holdings/{id}/trend/	Price, value, gain/loss, risk badge and confidence, direction and confidence, model versions, price chart with buy price line
Stock lookup	GET market/prices/{ticker}/ and history/	Latest price and recent history for any stored ticker

12.8 Screen inventory — analyst

Screen	API	Content
Home	GET dashboard/analyst/	Client count, total value, high-risk holdings list
Clients' portfolios	GET portfolios/	Only assigned investors' portfolios, read-only
Portfolio and holding detail	Same as investor	No add, edit or delete controls
Recompute prediction	POST analytics/holdings/{id}/recompute/, GET analytics/tasks/{task_id}/	Button on holding detail; shows progress until the task finishes, then reloads
Refresh prices	POST market/refresh/{ticker}/	Button on holding detail; shows a message when queued; handles 429

12.9 Screen inventory — admin

Screen	API	Content
Home	GET dashboard/admin/	Users by role, portfolio and holding counts, tickers tracked, latest price date, models in use with test metrics
Users	GET accounts/users/	Paginated list with role badges and assigned analyst
Edit user	PATCH accounts/users/{id}/	Role picker; analyst picker for investors; own role locked
All portfolios	GET portfolios/	Every portfolio with owner
Data and training	Opens /datasets/ and /ml/ in the browser	Upload and training stay in the Django web pages

12.10 Page design requirements

ID	Requirement
UI-01	Material 3 with one seed colour and light and dark themes.
UI-02	Designed for phone width first; layouts also work on tablets and in Chrome.
UI-03	Bottom navigation per role — investor: Home, Portfolios, Stocks, Profile; analyst: Home, Clients, Profile; admin: Home, Users, Portfolios, Profile.
UI-04	Same colour meaning as the web pages — risk: low green, medium amber, high red, not rated grey; direction: up green, down red. Labels are always shown as text too.
UI-05	Money uses two decimals and a currency symbol; gains show + in green and losses − in red.
UI-06	Every screen has loading, empty, error-with-retry and success states.
UI-07	Lists support pull-to-refresh and load more pages using the next link.
UI-08	Forms show the API's field errors under each field and disable the submit button while sending.
UI-09	Delete actions ask for confirmation.
UI-10	Predictions show confidence, model version and last prediction time, with a note that they are not financial advice.
UI-11	Values not yet available (no price, no prediction) show a clear placeholder such as 'Not rated yet'.

12.11 Acceptance criteria for the Flutter test

- The full journey works in the app: register an investor, admin promotes another user to analyst and assigns them, investor creates a portfolio and adds a holding, risk and forecast appear after predictions run, the analyst sees the portfolio read-only and can recompute, the token expires and refreshes without the user noticing, logout works.
- An investor never sees another investor's data; an analyst cannot edit; admin screens are hidden from other roles.
- The app works on the Android emulator, a physical phone and Chrome.

13. Celery and Redis Task Design

13.1 Redis databases

DB	Use
0	Celery broker
1	Celery results
2	Django cache and throttle counters
3	Refresh-token blacklist

Settings refuse to start if the four URLs are not different.

13.2 Scheduled and on-demand tasks

Task	Trigger	Purpose
core.tasks.ping	Every 15 minutes	Worker health check
market_data.tasks.fetch_daily_prices	Mon–Fri 22:00 UTC; Fetch now button	Download missing daily prices
market_data.tasks.refresh_single_ticker	Refresh API; new holding	One ticker, retries with back-off
market_data.tasks.import_dataset_upload	Datasets page	Import uploaded files with progress
analytics.tasks.recompute_holding_risk	Mon–Fri 23:00 UTC	Risk predictions for all holdings
analytics.tasks.recompute_holding_forecast	Mon–Fri 23:15 UTC	Direction predictions for all holdings
analytics.tasks.recompute_single_holding	Recompute API	Both predictions for one holding
ml_training.tasks.run_training	ML Training page	Train the chosen models
ml_training.tasks.retrain_models	Sunday 02:00 UTC	Weekly retraining recorded as a TrainingRun

13.3 Caching

Data	TTL	Cleared when
Portfolio summary, portfolio risk summary	300 seconds	Portfolio or holding changes, new prices for a held ticker, new predictions
Investor, analyst and admin dashboards	300 seconds	The same events, plus registrations, role and assignment changes, training runs

14. Machine Learning Pipeline

14.1 Code location

File (ml_training/)	Contents
algorithms.py	RandomForestClassifier, LogisticRegression, HistGradientBoostingClassifier and their settings
feature_engineering.py	The eight features
labels.py	Risk and direction labels
dataset.py	Loading prices and the time-based split
train_risk.py, train_forecast.py	Training, evaluation and baseline comparison
registry.py	Saving, versioning and loading models
services.py, tasks.py, views.py	Training runs from the web page and the weekly schedule

14.2 Features

Calculated per ticker using only past prices, so adding new prices never changes earlier values: daily_return, volatility_30, volatility_90, max_drawdown_90, price_to_ma_20, price_to_ma_50, rsi_14, volume_trend.

14.3 Model 1 — Risk classification

- Label: standard deviation of daily returns over the next 30 trading days, split into low / medium / high at the training period's 33rd and 66th percentiles.
- Model: RandomForestClassifier. Baseline: LogisticRegression, compared on macro F1.
- Output: risk_label and risk_score (probability of the predicted class).

14.4 Model 2 — Direction forecast

- Label: up if the next trading day's close is above today's close, otherwise down.
- Model: HistGradientBoostingClassifier. Baselines: always up, and same direction as today.
- Output: predicted_direction and prediction_confidence.

14.5 Training method and evaluation

Split	Dates
Train	Up to 31 December 2022
Validation	2023
Test	2024 onwards

- The risk split leaves a 45-day gap before each period so 30-day labels never overlap the next period.
- Metrics: accuracy, macro F1, per-class precision/recall/F1 and confusion matrix for risk; accuracy, F1 and ROC-AUC for direction.
- Next-day direction accuracy of about 50–55% is normal and is reported honestly against the baselines.

14.6 Versioning

Each run saves ml_models/<risk|forecast>/<version>.joblib with a JSON metadata file (features, date windows, label thresholds, parameters, metrics, baseline comparison). Versions look like risk-rf-2026.11.07-01. Predictions always use the newest version, and each holding stores the version that produced it.

15. Dataset Source and Ingestion

The historical data comes from the Kaggle dataset *S&P 500 Stocks, Daily Historical Data (10 Years)*: one CSV per ticker with Date, Open, High, Low, Close and Volume. Each row maps directly onto a price_history row, and ten years of daily data covers the training (to 2022), validation (2023) and test (2024 onwards) periods with enough rows for 90-day features and 30-day labels.

- Uploaded through the Datasets page (Section 11.1); no command line is needed.
- Invalid rows (bad dates, non-positive prices, high below low) are skipped and counted.
- Daily updates after the upload come from Yahoo Finance.

16. Environment Configuration

Variable	Purpose
DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT	PostgreSQL connection
SECRET_KEY, JWT_SECRET_KEY, JWT_REFRESH_SECRET_KEY	Three different secrets
ACCESS_TOKEN_LIFETIME_MINUTES, REFRESH_TOKEN_LIFETIME_DAYS	15 and 7
CELERY_BROKER_URL, CELERY_RESULT_BACKEND	Redis databases 0 and 1
REDIS_URL, TOKEN_BLACKLIST_REDIS_URL	Redis databases 2 and 3
CACHE_TTL_SECONDS	Summary cache lifetime (300)
LOGIN_THROTTLE_RATE, REGISTER_THROTTLE_RATE, MARKET_REFRESH_THROTTLE_RATE	5/minute, 10/hour, 10/hour
MARKET_SEED_TICKERS	Tickers always included in daily price updates

Fixed settings in settings.py: ML_MODELS_DIR (ml_models/), MEDIA_ROOT (media/), DATA_UPLOAD_MAX_NUMBER_FILES = 1000 (so a full S&P 500 folder can be uploaded), TIME_ZONE = UTC, and ALLOWED_HOSTS / CORS_ALLOWED_ORIGINS as in Section 12.3.

17. Project Directory Structure

```
stock_risk_engine/
├── accounts/          user model, JWT, auth APIs and pages, role management
├── analytics/         prediction services, tasks and APIs
├── core/              logger, decorators, responses, permissions, cache, throttles, fields
├── dashboards/        dashboard APIs and pages, portfolio and holding pages
├── market_data/       price history, dataset uploads, Yahoo Finance, price APIs
├── ml_training/       algorithms, features, labels, training, registry, training pages
├── portfolios/        portfolio and holding models, APIs, summary
├── stock_risk_engine/ settings.py, urls.py, celery.py
├── templates/         base, accounts, core, dashboards, market_data, ml_training
├── script/js/         page scripts and price chart
├── ml_models/         saved models (git-ignored)
├── media/             uploads while they are imported (git-ignored)
├── logs/              app.log (git-ignored)
├── .env.example
├── requirements.txt
└── stock_risk_engine.postman_collection.json

stock_risk_engine_flutter/   (separate project, Section 12)
└── lib/  config/  core/  models/  services/  screens/{common,investor,analyst,admin}/  widgets/
```

18. Security and Non-Functional Requirements

Area	Requirement
Data isolation	Querysets are scoped by role; outside records return 404
Tokens	Short-lived access tokens, separate secrets, refresh blacklist, invalidation on password change; Flutter stores tokens in secure storage
Rate limiting	Login, registration and price refresh limits with retry_after_seconds
Uploads	Admin only; .csv and .zip only; file size limits; files deleted after import; zip entries read without extracting to arbitrary paths
ML outputs	Written only by background tasks; never accepted from clients
Performance	Joined and annotated queries, Redis caching of summaries and dashboards, background processing of slow work
Observability	Rotating logs/app.log with logins, denials, cache activity, uploads, training runs, task results and timings
Environment	Local development with DEBUG = True; production settings (HTTPS, environment-driven hosts) only when deploying (Section 19)

19. Delivery Roadmap and Future Scope

Phase	Scope	Status
0 – 9	Foundation, accounts, core, market data, portfolios, ML, analytics, dashboards, hardening, testing	Complete
9B	GUI dataset upload and training, ml_training app, config merged into core	Complete
10	Flutter readiness on the backend (Section 12.3)	Next
11	Flutter test client (Sections 12.4 – 12.11)	Planned
12	Docker deployment: Gunicorn web, Celery worker and Beat, PostgreSQL, Redis, shared ml_models and media volumes	Planned
13	Future scope (below)	Planned

- Push notifications (Firebase Cloud Messaging) when a holding's risk label changes.
- Live price updates with WebSockets (Django Channels).
- Sector analytics and diversification score.
- Rebalancing suggestions based on composition and risk.
- LSTM forecasting as a comparison against Gradient Boosting.
- Choosing an older model version as the active one.

Appendix A — Sample Flutter Test Flow

Step	Who	Action	API / page
1	Admin	Upload the Kaggle zip and wait for completion	Web: /datasets/
2	Admin	Train both models with 'update predictions'	Web: /ml/
3	Investor (Flutter)	Register, then log in	POST register/, POST login/
4	Analyst (Flutter)	Register (becomes an investor)	POST register/
5	Admin (Flutter)	Promote the analyst and assign them to the investor	PATCH users/{id}/ ×2
6	Investor (Flutter)	Create a portfolio and add AAPL	POST portfolios/, POST portfolios/{id}/holdings/
7	Analyst (Flutter)	Open the client's holding and tap Recompute	POST recompute/, poll tasks/{id}/
8	Investor (Flutter)	See risk, forecast and price chart	GET risk/, forecast/, trend/
9	Investor (Flutter)	Keep using the app past 15 minutes; token refreshes silently	POST token/refresh/
10	Investor (Flutter)	Log out; the old refresh token is rejected	POST logout/

Appendix B — Machine Learning Disclaimer

Risk labels and direction forecasts are statistical estimates based on historical prices. They are shown with their confidence and model version and are not financial advice. Next-day direction accuracy near 50% is expected for daily stock data.